

Object Oriented Programming

MUSANINYANGE M.Larisse



Object–Oriented Programming

- object-oriented concepts
 - classes
 - objects
 - encapsulation
 - inheritance
 - Polymorphism
 - Interface and abstract



What is OOP?

Object-oriented programming (OOP) is a computer programming model that organizes software design around data, or objects, rather than functions and logic. An object can be defined as a data field that has unique attributes and behavior. OOP focuses on the objects that developers want to manipulate rather than the logic required to manipulate them.

Procedural programming is about writing procedures or methods that perform operations on the data, while object-oriented programming is about creating objects that contain both data and methods



Advantages of OOP

Object-oriented programming has several advantages over procedural programming:

- ❖ OOP is faster and easier to execute
- ❖ OOP provides a clear structure for the programs
- ❖ OOP helps to keep the Java code DRY "Don't Repeat Yourself", and makes the code easier to maintain, modify and debug
- ❖ OOP makes it possible to create full reusable applications with less code and shorter development time

Tip: The "Don't Repeat Yourself" (DRY) principle is about reducing the repetition of code. You should extract out the codes that are common for the application, and place them at a single place and reuse them instead of repeating it.



Classes

- a class is a template for objects, and an object is an instance of a class.

When the individual objects are created, they inherit all the variables and methods from the class.

- Classes and objects are the two main aspects of object-oriented programming.

Look at the following illustration to see the difference between class and objects:



What are Classes and Objects?

class

Fruit

objects

Apple

Banana

Mango



```
class ClassName {  
    // class body  
}
```

There are three major components of class in Java.

1) Variable/attribute

Variable is a reserved memory location to hold a value. In other words, when we want to store any value and give it a name, that value takes a reserved space in memory and it's called as a variable.



► 2) Method

A method is a block of code which only runs when it is called. You can pass data, known as parameters, into a method. Methods are used to perform certain actions, and they are also known as functions.



►3) Constructor

A constructor is like a special method in Java, which is used to initialize an object of a Java class and Constructor is very important for every Java class and if we don't declare the constructor, the compiler creates a default constructor of a Java class.

► A constructor must be the same name of a Java class. It does not have a return type.



Types of constructors

- No-argument constructor
- Parameterized constructor
- Default constructor



No-argument constructor

- Has zero parameters
- Example : class student with no arg-contrsructor

```
class Student{  
    private String name;  
    Student () {  
        name = "kamana";  
    }  
}
```



Parameterized/explicit constructor

- Has one or more parameters
- Eg. Student class with a parameterized constructor

```
class Student{  
    private String name;  
    Student (String name) {  
        this.name = name;  
    }  
}
```



Default constructor

- Provided by the compiler when no constructor was declared
- Example - Student class with no constructor



Default constructor

```
class Student{  
}
```



```
class Student{  
    Student() {  
    }  
}
```



Parameterized methods and constructors

- Information can be passed to methods as parameter. Parameters act as variables inside the method.
- Parameters are specified after the method name, inside the parentheses. You can add as many parameters as you want, just separate them with a comma.



Method overloading

➤ multiple methods can have the same name with different parameters

Eg:

```
int myMethod(int x)
float myMethod(float x)
double myMethod(double x, double y)
```



Constructor overloading

➤ Declaring more than one constructor inside same class

Eg:

```
1.  public class Student
2.  {
3.      String name;
4.      int rollNo;
5.      String className;
6.      boolean isFromCity;
7.
8.      Student(String name, int rollNo)
9.      {
10.         this.name = name;
11.         this.rollNo = rollNo;
12.     }
13.
14.     Student(String name, int rollNo, String className, boolean isFromCity)
15.     {
16.         this.name = name;
17.         this.rollNo = rollNo;
18.         this.className = className;
19.         this.isFromCity = isFromCity;
20.     }
21. }
```



“Static” keyword

- The **static** keyword is used to declare a members that is associated with the class, not with individual instances of that class. Static variables are used to store "global" information about a class (for example, a static variable could be used to maintain the total number of Student objects created).
- Static members exist even if no instance of their class is created.



Example of static keyword

```
class Test
{
    // static method
    static void m1()
    {
        System.out.println("from m1");
    }

    public static void main(String[] args)
    {
        // calling m1 without creating
        // any object of class Test
        m1();
    }
}
```



Example of static keyword

```
class Test
{
    // static variable
    static int a = 10;
    static int b;

    // static block
    static {
        System.out.println("Static block initialized.");
        b = a * 4;
    }

    public static void main(String[] args)
    {
        System.out.println("from main");
        System.out.println("Value of a : "+a);
        System.out.println("Value of b : "+b);
    }
}
```



Java this keyword

The **this** keyword refers to the current object in a method or constructor.

The use of the **this** keyword is to eliminate the confusion between class attributes and parameters with the same name



Java this keyword example

```
public class Main {  
    int x; // Constructor with a parameter  
  
    public Main(int x) {  
        this.x = x; } // Call the constructor  
  
    public static void main(String[] args) {  
  
        Main myObj = new Main(5);  
        System.out.println("Value of x = " + myObj.x);  
  
    }  
}
```

If you omit the keyword in the example above, the output would be "0" instead of "5".



Object

- What is an object?
- Instance of a class, i.e, it is created from a class
- Eg. Objects: Student John, Student Alice, ...



Object creation/Instantiation

- **new** is a Java keyword which tells the JVM to create an new object of a given class type

```
Coordinate c = new Coordinate(2, 4);  
String s = new String("Hello");  
Integer i = new Integer(3);
```

- It's always followed by a constructor (default, explicit or implicit), which it calls to create the object



Reference vs Object

- When we create an object, we usually assign a reference to it to a variable, i.e.

```
Coordinate c = new Coordinate(2,5);
```

- c is a variable – a reference to the object which now exists in memory. We can assign it a new different object...

```
c = new Coordinate(8, -2);
```

- and then the first one will be lost!



Pillars of OOP

OOP moves around its four most important pillars. These are:

- Inheritance,
- Polymorphism,
- Abstraction,
- and Encapsulation.



Duplication = Wasted time

- Sometimes a class would require some of the same variables and methods as another class
- Duplicating code is **ALWAYS** bad
- Programmers needed a way to share members between different classes, so someone invented ***inheritance***



Duplication = Wasted time

- Supposing we created a set of classes for different shapes. Much of our code might be duplicated...

```
class Square {  
    int color  
    int width  
    int height
```

```
getColor() {..  
}
```

```
class Circle {  
    int color  
    int radius
```

```
getColor() {..  
}
```

```
setColor() {..  
}
```

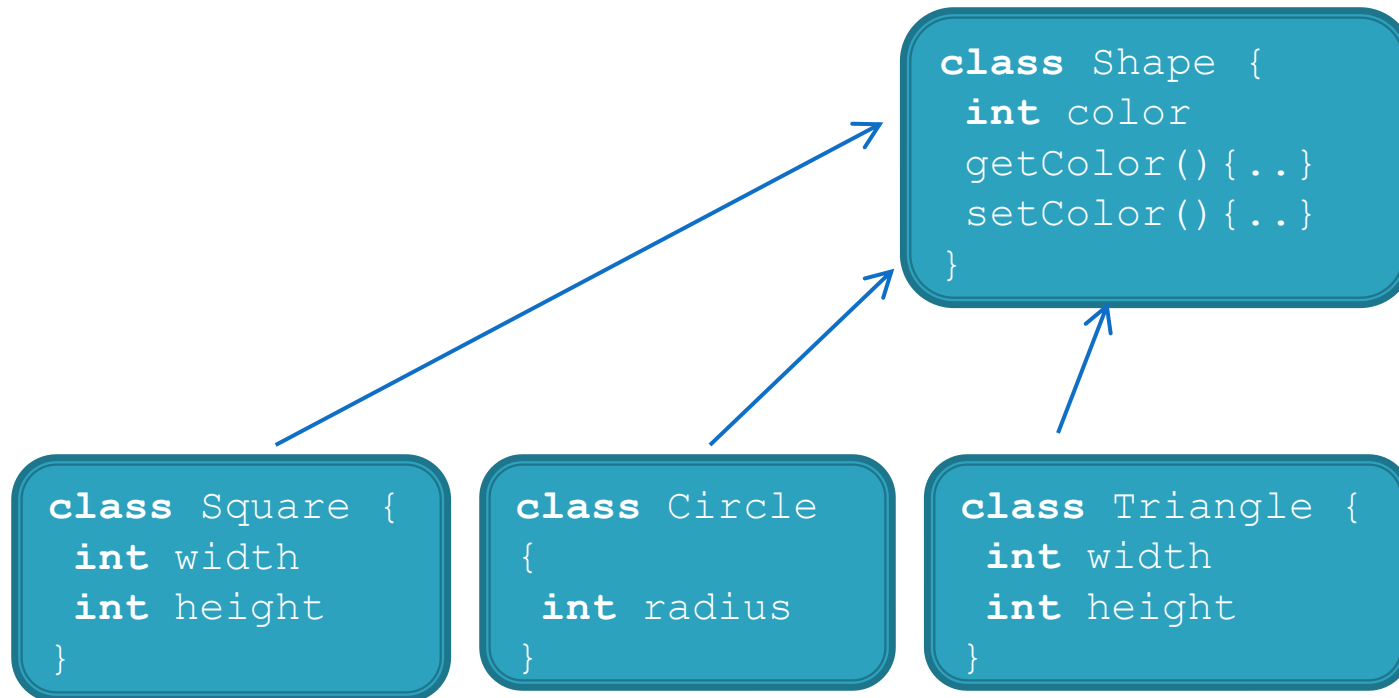
```
class Triangle {  
    int color  
    int width  
    int height
```

```
getColor() {..  
}
```



Inheritance to the rescue!

- We start by extracting all the variables and methods that are *common* to all shapes



Extending a class

- Then we use the **extends** keyword (in Java) to tell the compiler that *Square*, *Circle* and *Triangle* should inherit all the functionality from *Shape*

```
class Shape {  
    int color  
    getColor(){..  
    setColor(){..  
}
```

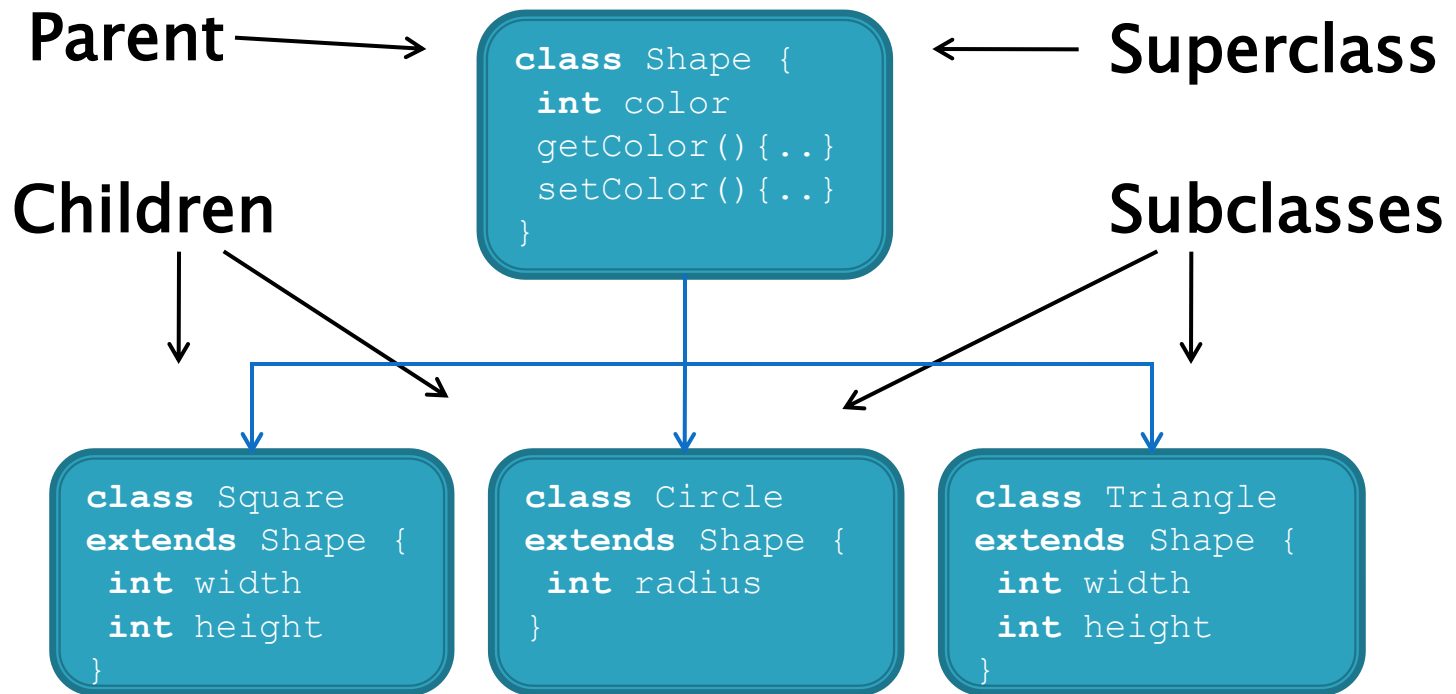
```
class Square  
extends Shape {  
    int width  
    int height  
}
```

```
class Circle  
extends Shape {  
    int radius  
}
```

```
class Triangle  
extends Shape {  
    int width  
    int height  
}
```



A Family Tree



Circle *is a* Shape

- Circle **is a** Shape, so we can do this...

```
Shape c = new Circle();  
int i = c.getColor();
```

- The object created by the JVM is of type `Circle`, but we can reference it with variable of type `Shape`



But Shape *is not a* Circle!

- Inheritance goes one way – Shape is not a kind of Circle, so this won't compile...

```
Circle c = new Shape(); // Error!!!!
```

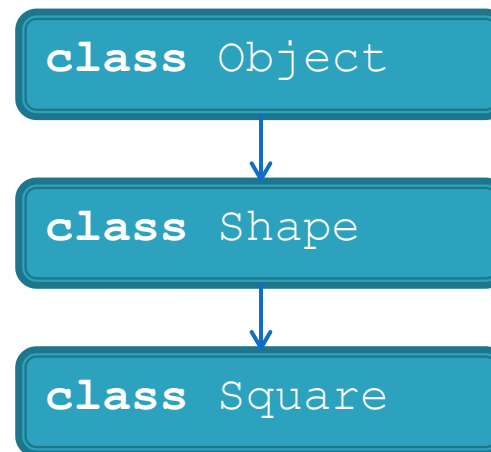
- And this definitely won't work (Circle and Triangle are siblings)

```
Circle c = new Triangle(); // Error!!!!
```



Every object is an Object...

- Anytime you create a class in Java, it automatically inherits from a class called Object
- So every class is related!



```
Object o = new Shape(); // Works!!
```



Casting Objects...

- We can also cast objects as long as the object being cast is what we're casting it to!

```
Square q = new Square();
```

```
Shape s = q;           // Implicit cast
```

```
Shape t = (Shape)q;    // Explicit cast
```

- The above code works because **Square is a Shape**



Casting Objects...

- But what if the reference type isn't exactly the same as the object type?
- The compiler will throw an error if you try an implicit cast
- But the JVM will do the cast, as long as the actual object is of the right type...

```
Shape s = new Square();  
Object o = new Square();  
  
Square t = s;           // Compiler Error  
Square q = (Square)s;   // Works!!!  
Square r = (Square)o;   // Works!!!
```



Super keyword

- The super keyword in java is a reference variable that is used to refer parent class objects. The keyword “super” came into the picture with the concept of Inheritance. Basically this form of super is used to initialize superclass variables when there is no constructor present in superclass.

- Ex:

```
class Vehicle {
```

```
    // Attribute
```

```
    int maxSpeed = 120;
```

```
}
```

```
// Class 2
```

```
// sub class Car extending vehicle
```

```
class Car extends Vehicle {
```

```
    int maxSpeed = 180;
```

```
    // Method
```

```
    void display()
```

```
{
```

```
    // Printing maxSpeed of parent class (vehicle) as
```

```
    // super keyword refers to parent class
```

```
    System.out.println("Maximum Speed: "
```

```
        + super.maxSpeed);
```

```
}
```

```
}
```



Super keyword

```
// Class 3
// Main class
class GFG {

    // Main driver method
    public static void main(String[] args)
    {
        // Creating an object of child class
        Car small = new Car();

        // Calling out method defined inside child class
        small.display();
    }
}
```

- The super keyword can also be used to access the parent class constructor by adding '()' after it, i.e. super(). Also do remember that 'super()' can call both parametric as well as non-parametric constructors depending upon the situation.



Super keyword

```
// Java code to demonstrate super()

// Class 1
// Helper class
// Parent class - Superclass
class Person {

    // Constructor of superclass
    Person()
    {
        // Print statement of this class
        System.out.println("Person class Constructor");
    }
}

// Class 3
// Main class
class GFG {

    // Main driver method
    public static void main(String[] args)
    {
        // Creating object of subclass
        // inside main() method
        Student s = new Student();
    }
}
```

```
// Class 2
// Helper class
// Subclass extending the above superclass
class Student extends Person {
    Student()
    {
        // Invoking the parent class constructor
        // with the usage of super() word
        super();

        // Print statement whenever subclass constructor is
        // called
        System.out.println("Student class Constructor");
    }
}
```

Output:

```
Person class Constructor
Student class Constructor
```



Polymorphism

- The word polymorphism means having many forms. In simple words, we can define polymorphism as the ability of a message to be displayed in more than one form.
- **Real-life Illustration Polymorphism:** A person at the same time can have different characteristics. Like a man at the same time is a father, a husband, and an employee. So the same person possesses different behavior in different situations. This is called polymorphism.
- polymorphism allows you to define one interface and have multiple implementations.



Static polymorphism(compile-time polymorphism /method overloading)

- Having more than one function with the same name in one class having a different prototype.

```
// Java program to demonstrate
// compile-time polymorphism
public class GFG {

    // First addition function
    public static int add(int a, int b)
    {
        return a + b;
    }

    // Second addition function
    public static double add(
        double a, double b)
    {
        return a + b;
    }
}
```



Dynamic polymorphism(runtime polymorphism /method overriding)

- Method overriding says the child class has the same method as declared in the parent class. It means if the child class provides the specific implementation of the method that has been provided by one of its parent classes then it is known as method overriding.

```
// Java program to demonstrate  
// runtime polymorphism
```

```
// Implementing a class  
class Test {
```

```
    // Implementing a method  
    public void method()  
    {  
        System.out.println("Method 1");  
    }  
}
```

```
// Defining a child class  
public class GFG extends Test {
```

```
    // Overriding the parent method  
    public void method()  
    {  
        System.out.println("Method 2");  
    }  
}
```



Modifiers

- In Java, **modifiers** are keywords used to **define the scope, behavior, and accessibility** of classes, variables, methods, and constructors.
- They are broadly divided into two categories:
- **Access Modifiers**
- **Non-Access Modifiers**



Access Modifiers

- Access modifiers **control visibility** (who can access a class, method, or variable).



Access Modifiers: Class access

- What does it mean to access a class? When we say code from one class (class A) has access to another class (class B), it means class A can do one of three things:
 - Create an instance of class B.
 - Extend class B (in other words, become a subclass of class B).
 - Access certain methods and variables within class B, depending on the access control of those methods and variables.
- For classes, you can use either **public** or **default**



Access Modifiers

Class Access: 1. Default access/Package access

A class with default access has no modifier preceding it in the declaration! It's the access control you get when you don't type a modifier in the class declaration.

- Think of default access as package-level access, because a class with default access can be seen only by classes within the same package.
- For example, if class A and class B are in different packages, and class A has default access, class B won't be able to create an instance of class A, or even declare a variable or return type of class A.
- In fact, class B has to pretend that class A doesn't even exist, or the compiler will complain.



Access Modifiers

Class Access: 1. Default access

- Example

```
package rw.ac.rca.hrms;  
class Person{  
}
```

```
package rw.ac.rca.sms;  
import rw.ac.auca.hrms.Person;  
class Student extends Person{  
}
```

- For this example you get an error, because a class declared with default access can not be accessed outside its package.



Access Modifiers

Class Access: 2. Public access

- A class declaration with the public keyword gives all classes from all packages access to the public class.
- In other words, all classes in the Java Universe (JU) have access to a public class.
- Don't forget, though, that if a public class you're trying to use is in a different package from the class you're writing, you'll still need to import the public class.
- To make the code work, we need to add the keyword public in front of the superclass declaration.



Class member access modifiers

- In some cases, we may or not be allowed to directly access some of the instance variables or methods for an object.
- Method and variable members are usually given access control in exactly the same way.
- For example, an instance variable declared as private in some class is only accessible by the methods defined inside that class. Such instance variables are similar to device parameters that cannot be accessed directly from a remote control.
- For example, some devices have internal parameters that can only be read or assigned by a factory technician (and a user is not allowed to change those parameters without violating the device's warranty).



Class member access modifiers

- If class A has access to a member of class B, it means that class B's member is visible to class A. When a class does not have access to another member, the compiler will slap you for trying to access something that you're not even supposed to know exists!
- Whereas a class can use just two of the four access control levels (default or public), members can use all four:
 - public (+)
 - protected (#)
 - default ()
 - private (-)



Class member access modifiers

Visibility	Public	Protected	Default	Private
From the same class	Yes	Yes	Yes	Yes
From any class in the same package	Yes	Yes	Yes	No
From a subclass in the same package	Yes	Yes	Yes	No
From a subclass outside the same package	Yes	Yes, <i>through inheritance</i>	No	No
From any non-subclass class outside the package	Yes	No	No	No



Non Access Modifiers

Class level

- You can modify a class declaration using the keyword **final**, or **abstract**.
- These modifiers are in addition to whatever access control is on the class, so you could, for example, declare a class as both public and final.
- But you can't always mix nonaccess modifiers.
- You must never, ever, ever mark a class as both **final** and **abstract**.



Non Access Modifiers :1. Final keyword

- When used in a class declaration, the final keyword means the class can't be subclassed. In other words, no other class can ever extend (inherit from) a final class, and any attempts to do so will give you a compiler error.
- So why would you ever mark a class final? After all, doesn't that violate the whole object-oriented (OO) notion of inheritance? You should make a final class only if you need an absolute guarantee that none of the methods in that class will ever be overridden.
- If you're deeply dependent on the implementations of certain methods, then using final gives you the security that nobody can change the implementation out from under you.



Non Access Modifiers :2. Abstract keyword

- An abstract class can never be instantiated.
- Its sole purpose, mission in life, raison d'être, is to be extended (subclassed). (Note, however, that you can compile and execute an abstract class, as long as you don't try to make an instance of it.)
- Example:

```
abstract class Person{  
}  
class Student extends Person {  
}
```



Nonaccess Member Modifiers:1. Final

- The final keyword prevents a method from being overridden in a subclass, and is often used to enforce the API functionality of a method.
- A final instance variable is one that must be assigned an initial value, and then can never be assigned a new value after that. If it is a base type, then it is a constant
- If an object variable is final, then it will always refer to the same object (even if that object changes its internal state).



Nonaccess Member Modifiers:2. Static

- The static keyword is used to declare a members that is associated with the class, not with individual instances of that class. Static variables are used to store "global" information about a class (for example, a static variable could be used to maintain the total number of Student objects created).
- Static members exist even if no instance of their class is created.



Nonaccess Member Modifiers:3. abstract

- Can only be used in an abstract class, and can only be used on methods. The method does not have a body, for
- example `abstract void run();`. The body is provided by the subclass (inherited from).



Encapsulation

- ▶ Encapsulation means putting together all the variables (instance variables) and the methods into a single unit called Class.
- ▶ It also means hiding data and methods within an Object.
- ▶ Encapsulation provides the security that keeps data and methods safe from inadvertent changes.
- ▶ In Java, encapsulation is achieved by declaring the instance variables of a class as private, which means they can only be accessed within the class.
- ▶ To allow outside access to the instance variables, public methods called getters and setters are defined, which are used to retrieve and modify the values of the instance variables, respectively.



Encapsulation example

```
class Person {
    private String name;
    private int age;

    public String getName() { return name; }

    public void setName(String name) { this.name = name; }

    public int getAge() { return age; }

    public void setAge(int age) { this.age = age; }
}

public class Main {
    public static void main(String[] args)
    {
        Person person = new Person();
        person.setName("John");
        person.setAge(30);

        System.out.println("Name: " + person.getName());
        System.out.println("Age: " + person.getAge());
    }
}
```



Benefits of encapsulation

- **Data Hiding / Abstraction**. A class can have total control over what is stored in its fields
- **Makes the class simpler to use** as the users of a class do not need to know how the class stores its data. A class can change the field and users of the class do not need to change any of their code.
- The fields of a class can be made **read-only or write only**



Abstraction

- In Java, abstraction is achieved by **interfaces** and **abstract classes**.
- Data Abstraction may also be defined as the process of identifying only the required characteristics of an object ignoring the irrelevant details.



Abstract Class

```
abstract class Shape
{
    int color;
    // An abstract function
    abstract void draw();
}
```



Abstract Class rules

- An instance of an abstract class can not be created.
- Constructors are allowed.
- There can be a final method in abstract class but any abstract method in class (abstract class) can not be declared as final or in simpler terms final method can not be abstract itself as it will yield an error: “Illegal combination of modifiers: abstract and final”
- We can define static methods in an abstract class
- If a class contains at least one abstract method then compulsory should declare a class as abstract
- If the Child class is unable to provide implementation to all abstract methods of the Parent class then we should declare that Child class as abstract so that the next level Child class should provide implementation to the remaining abstract method



Abstract class example

```
import java.io.*;

abstract class Subject {
    Subject() {
        System.out.println("Learning Subject");
    }

    abstract void syllabus();

    void Learn(){
        System.out.println("Preparing Right Now!");
    }
}

class IT extends Subject {
    void syllabus(){
        System.out.println("C , Java , C++");
    }
}
```



Abstract class example

```
class GFG {  
    public static void main(String[] args) {  
        Subject x=new IT();  
  
        x.syllabus();  
        x.Learn();  
    }  
}
```



Interface

- An interface in Java is usually referred to as the “Blueprint of a Class”. This means that by looking at an interface, we can predict the behavior of the classes that will implement this interface.
- Interfaces are used to achieve abstraction in Java. Abstraction is one of the major pillars of Object-Oriented Programming Systems (OOPS).
- If we go into the definition of an interface, we can say that an interface in Java is an abstract type used to show the behavior of a class as it acts like a blueprint for the classes that implement it
- The classes are inherited whereas, the interfaces are implemented.



Interface

- An **interface** in Java is a special type that defines a **contract** (blueprint) for classes.
- It can contain:
 - **Abstract methods** (implicitly public and abstract).
 - **Default methods** (with body, since Java 8).
 - **Static methods** (with body, since Java 8).
 - **Private methods** (with body, since Java 9, used as helpers inside the interface).
- All **data members (variables)** declared in an interface are **public static final** by default. This means they are **constants** and must be initialized when declared.
- An interface **cannot be instantiated directly** because it has no complete implementation.
Instead:
 - A class must **implement** the interface to provide method bodies.
 - Or, we can use **anonymous inner classes** or **lambdas** (for functional interfaces).



Why use interfaces ?

- To achieve loose coupling (reduce dependency on specific implementations)
- To support multiple inheritance (since Java does not allow multiple class inheritance)
- To define a common standard for different classes.

Example: Payment systems (PayPal, MasterCard, MoMo) can all implement the same PaymentGateway interface.



Evolution of Interfaces in Java

- **Java before 8 (Java 7 and earlier)**
- Interfaces could only have:
 - **Abstract methods** (no body).
 - **Public static final variables** (constants).
- Example :

```
interface Vehicle {  
    void start(); // abstract method  
    void stop();  
}
```

```
class Car implements Vehicle {  
    public void start() { System.out.println("Car starts..."); }  
    public void stop() { System.out.println("Car stops..."); }  
}
```



Evolution of Interfaces in Java ctd”

- Limitation: If you add a new method in the interface, **all implementing classes break** (they must implement it).



Evolution of Interfaces in Java(Java 8)

- Java 8 introduced:
- **Default methods** → methods with body inside interface.
 - To add new functionality **without breaking existing classes**.
- **Static methods** → utility/helper methods inside interface/called without creating an object
- **Functional Interfaces** → interfaces with exactly one abstract method (used in Lambda Expressions).



Evolution of Interfaces in Java(Java 8)

- Example:

```
package com.year2;

public interface PaymentGateway {
    void pay(double amount); // abstract

    // Default method
    default void printReceipt() {
        System.out.println("Receipt generated...");
    }

    // Static method
    static void showGatewayRules() {
        System.out.println("All payments must be secure.");
    }
}

class PayPal implements PaymentGateway {
    public void pay(double amount) {
        System.out.println("Paying " + amount + " via PayPal.");
    }
}
```

```
package com.year2;
```

```
public class Main {
```

```
    public static void main(String[] args) {
        // TODO Auto-generated method stub
```

```
        PaymentGateway pg = new PayPal();
        pg.pay(100);
        pg.printReceipt(); // default method
        PaymentGateway.showGatewayRules(); // static method
```

```
    }
```

9/26/2025

Evolution of Interfaces in Java(Java 9)

- Java 9 introduced **private methods in interfaces**.
- Purpose: To **avoid code duplication** inside default and static methods.

```
package com.year2;

public interface Logger {
    default void logInfo(String msg) {
        log("INFO", msg);
    }
    default void logError(String msg) {
        log("ERROR", msg);
    }

    // Private helper method
    private void log(String level, String msg) {}
    System.out.println "[" + level + " ] " + msg;
}
}
```



Interface example

```
import java.util.*;
class Phone {
    void makeCall(String person) {
    }
    void sendMessage(String person) {
    }
}
interface Camera {
    void clickPicture();
    void recordVideo();
}
class SmartPhone extends Phone implements Camera{
    public void clickPicture() {
        System.out.println("Auto focus on human face");
    }
    public void recordVideo() {
        System.out.println("Auto removal of background noise");
    }
    public void accessInternet() {
        System.out.println("Woah! I'm smart enough to do this");
    }
}
```



Interface example 2

```
import java.util.*;

class Phone {
    void makeCall(String person) {
    }
    void sendMessage(String person) {
    }
}

interface Camera {
    int megaPixels = 48;
    void clickPicture();
    void recordVideo();
}

interface Calculator {
    void add(int x, int y);
    void subtract(int x, int y);
    void multiply(int x, int y);
    void divide(int x, int y);
}
```



Interface example 2

```
class SmartPhone extends Phone implements Camera, Calculator{
    public void clickPicture() {
        System.out.println("Auto focus on human face");
    }
    public void recordVideo() {
        System.out.println("Auto removal of background noise");
    }
    public void accessInternet() {
        System.out.println("Woah! I'm smart enough to do this");
    }
    public void add(int x, int y) {
        System.out.println(x + y);
    }
    public void subtract(int x, int y) {
        System.out.println(x - y);
    }
    public void multiply(int x, int y) {
        System.out.println(x * y);
    }
    public void divide(int x, int y) {
        System.out.println(x / y);
    }
}
```



Interface example 2

```
public class Main {  
    public static void main(String[] args) {  
        SmartPhone PrepPhone = new SmartPhone();  
        PrepPhone.clickPicture();  
        PrepPhone.add(10,20);  
    }  
}
```



Interface with default method

```
package Developer;

public interface Computer2 {
    void playMusic();

    default void runCode() {
        System.out.println(" in computer interface");
    }

    default void playVideo() {
        System.out.println(" in computer interface");
    }
}
```

```
package Developer;

public class Desktop implements Computer2 {

    public void runCode() {
        System.out.println("am runing codes using Desktop today")
    }

    @Override
    public void playMusic() {
        System.out.println("playing music with desktop");
    }

}
```

```
package Developer;

public class Laptop implements Computer2 {

    public void runCode() {
        System.out.println("am runing codes using laptop today");
    }

    @Override
    public void playMusic() {
        System.out.println("playing music with laptop");
    }

}
```

